

Regiões críticas nomeadas e travas explícitas

Para essa atividade, compreendi que é necessário realizar duas implementações:

1. Um programa que faz N inserções em DUAS listas, escolhendo aleatoriamente a cada iteração em qual lista será adicionado um valor, de forma paralela usando openMP.
2. Um programa que faz N inserções em M listas, escolhendo aleatoriamente a cada iteração em qual das M listas será adicionado um valor, de forma paralela usando openMP.

I. Lista ligada

Para ambas as implementações, foi necessário criar uma estrutura de linked [list](#). Segue a implementação utilizada para os dois códigos:

```
#include <stdio.h>
#include <stdlib.h>

// Nó da lista, mantém referência pro próximo e qual valor está guardando.
// next == NULL significa que chegou ao fim da lista.
struct Node {
    int val;
    struct Node *next;
} typedef Node;

// Inicializa um novo nó, retorna o ponteiro para o nó inicializado.
Node *new(int val) {
    Node *ptr = (Node *)malloc(sizeof(Node));

    ptr->val = val;
    ptr->next = NULL;

    return ptr;
}

// Varre a lista até o final, e ao fim adiciona um novo nó em last->next.
void append(Node **ptr, int val) {
    Node *last = new(val);

    if (*ptr == NULL) {
        *ptr = last;
        return;
    }

    Node *curr = *ptr;

    while (curr->next != NULL) {
        curr = curr->next;
    }

    curr->next = last;
}
```

```
// Enquanto houver itens, imprime eles na tela.
void print(Node *ptr) {
    printf("[");

    Node *curr = ptr;
    while (curr != NULL) {
        printf("%d, ", curr->val);
        curr = curr->next;
    }

    printf("]");
}
```

II. Implementação para duas listas

Para a primeira implementação, utilizamos uma região paralela por meio da diretiva `parallel`. Com isso, geramos a seed que será replicada para cada thread instanciada.

Após isso, iniciamos uma região em que apenas uma thread vai executar, por meio da diretiva `single`. Utilizamos ela para que apenas uma thread faça a iteração N vezes.

Dentro de cada iteração, criamos uma task para fazer o append na lista de forma aleatorizada. Como existem apenas duas listas, antes de cada operação de append, criamos uma região crítica para cada, dessa forma conseguimos fazer com que quaisquer duas threads consigam fazer o append das listas `l1` e `l2` sem barreiras.

Apenas duas threads que queiram fazer o append na mesma lista serão travadas por meio da região crítica.

```
#pragma omp parallel
{
    uint seed = (uint)omp_get_thread_num();

    #pragma omp single
    {
        for (int i = 0; i < N; i++) {

            int r = rand_r(&seed);
            #pragma omp task firstprivate(i, seed) shared(l1, l2)
            {
                if (r % 2 == 0) {
                    #pragma omp critical(l1_crit)
                    {
                        append(&l1, i * 3);
                    }
                } else {
                    #pragma omp critical(l2_crit)
                    {
                        append(&l2, i * 4);
                    }
                }
            }
        }
    }
}
```

III. Implementação para M listas

Para manter a mesma funcionalidade para M listas, é preciso criar uma lista de M locks. Como a quantidade de listas é dinâmica, não dá pra escrever o pragma omp critical M vezes dinamicamente em código. Por isso, precisamos criar uma lista de locks de tamanho M. Um para cada lista.

Dessa forma, conseguiremos fazer com que o lock i garanta a região crítica da lista i .

```
#include "ll.c"
#include <omp.h>
#include <stdio.h>
#include <stdlib.h>

int N = 200;
int M = 10;

int main() {

    // M listas
    Node **listas = (Node **)malloc(M * sizeof(Node *));
    // M locks
    omp_lock_t *locks = (omp_lock_t *)malloc(M * sizeof(omp_lock_t));

    for (int i = 0; i < M; i++) {
        listas[i] = NULL;
        // Inicializa os N locks
        omp_init_lock(&locks[i]);
    }

    #pragma omp parallel
    {
        uint seed = (uint)omp_get_thread_num();
        #pragma omp single
        {

            for (int i = 0; i < N; i++) {
                #pragma omp task firstprivate(i) shared(listas, locks, M, seed)
                {
                    int lista_escolhida = rand_r(&seed) % M;

                    omp_set_lock(&locks[lista_escolhida]);
                    append(&listas[lista_escolhida], i);
                    omp_unset_lock(&locks[lista_escolhida]);
                }
            }
        }
    }

    for (int i = 0; i < M; i++) {
        printf("\nLista %d: ", i);
        print(listas[i]);
    }

    return 0;
}
```